Diretrizes Fase 2 — Melhoria de Req. Não Funcionais do PictuRAS

Especificação, *refactoring* e avaliação

Versão 1

André Ferreira João M. Fernandes Miguel Regedor Paulo Sousa

Requisitos e Arquiteturas de Software

Mestrado em Engenharia Informática

Universidade do Minho

4 de novembro de 2026

Objetivo

Na Fase 1, o PictuRAS passou a suportar vídeo numa lógica de **MVP**: o foco foi a validação funcional, sem exigir o nível de qualidade não funcional de uma solução em produção. O objetivo da **Fase 2** é dar o passo seguinte: escolher **dois requisitos não funcionais** do sistema e melhorá-los por via de *refactoring*, **sem introduzir novas funcionalidades**.

Tal como definido no enunciado, a Fase 2 inclui a **definição dos requisitos não funcionais** a melhorar, a **reestruturação do código assistida por agentes de IA** e a **avaliação do impacto**, demonstrando a melhoria alcançada. O trabalho e o relatório devem, por isso, fechar o ciclo *requisito não funcional* → *quantificação* → *medição inicial* → *refactoring* → *medição final*.

> **Entrega da Fase 2: 20 de novembro de 2026**.
> **Defesas: 23–27 de novembro de 2026**.

Ponto de partida

A base da Fase 2 é a solução da Fase 1 selecionada pela equipa docente: a do grupo **PL1-G4**. A escolha deve-se, sobretudo, a dois fatores: a solução mantém a génese do PictuRAS (encadeamento de ferramentas e processamento em lote de todos os *media* do projeto) e apresenta ao utilizador as ferramentas como aplicáveis, sempre que possível, a imagem e a vídeo.

A solução de base é composta por:

- o **repositório oficial** da aplicação (`https://github.com/picturas-um/picturas-27`), onde o código do PL1-G4 já está integrado, juntamente com algumas correções feitas pela equipa docente e com duas funcionalidades de vídeo acrescentadas posteriormente: **legendagem automática** e **extração de fotogramas**;
- o documento de extensão de requisitos do PL1-G4 (`doc-ext-requisitos-2627.pdf`), a ler em conjunto com os documentos de requisitos e de solução arquitetural da base anterior;
- a documentação do repositório: o `README.md`, que já descreve o suporte a vídeo, e o `AGENTS.md`, com orientações para agentes de IA que trabalhem no código.

As funcionalidades de legendagem automática e de extração de fotogramas não estão cobertas pelo documento de requisitos do PL1-G4: o seu comportamento está descrito apenas no `README.md` e no `AGENTS.md`, e algumas das suas regras (e.g., a extração de fotogramas é sempre o último passo do pipeline) não têm um requisito explícito associado.

O *commit* do repositório oficial a partir do qual a equipa parte deve ser identificado no relatório.

Antes de escolher os requisitos não funcionais a melhorar, cada equipa deve **estudar a solução de base**, lançá-la e compreender, em particular, como é orquestrado o processamento em lote de imagens e de vídeos e como é tratada a diferença entre os dois tipos de *media*.

Princípios

- **Refactoring, não novas funcionalidades.** O comportamento funcional do sistema é preservado. Alterações visíveis ao utilizador só são aceitáveis quando são o próprio objeto da melhoria (e.g., em usabilidade) e não acrescentam funcionalidades novas.
- **Os requisitos são definidos pela equipa.** Este documento não fixa requisitos, métricas nem valores-alvo. Identificar o que melhorar, especificá-lo de forma quantitativa e justificar os valores escolhidos é parte central do trabalho.
- **Medir antes de mudar.** Cada requisito é medido na solução de base **antes** do *refactoring* e novamente no fim, com o mesmo método e nas mesmas condições. Uma melhoria sem medição inicial não é demonstrável.
- **Evidência reprodutível.** Os cenários, *scripts*, dados e ambiente das medições são entregues e descritos de forma que outra pessoa os consiga repetir.
- **Qualidade em vez de quantidade.** Na classificação, são valorizadas a **pertinência** e a **complexidade** dos requisitos não funcionais escolhidos. A pertinência mede-se pelo impacto real da fragilidade nos *stakeholders* do PictuRAS; a complexidade, pela exigência técnica do *refactoring* e da sua avaliação. Valoriza-se também a profundidade da análise, a adequação das decisões arquiteturais e a solidez da avaliação, mais do que a extensão das alterações.
- **IA documentada e validada.** Ferramentas, *skills*, tarefas apoiadas e validação feita pela equipa são documentadas, tanto no relatório como no código. A equipa é responsável por todo o conteúdo entregue.

Escolha dos requisitos não funcionais

- Cada equipa escolhe **dois requisitos não funcionais distintos**, classificados segundo a ISO/IEC 25010:2011 (característica e, quando aplicável, subcaracterística).
- A escolha deve partir de **fragilidades concretas** da solução de base, identificadas no código, na arquitetura ou em medições, e ser justificada pelo seu interesse para os *stakeholders* do PictuRAS.
- Os dois requisitos não funcionais podem estar relacionados (e.g., um deles ser condição para o outro), mas cada um é especificado, implementado e avaliado por si.
- A melhoria de um requisito não funcional pode ter pré-requisitos ou efeitos colaterais noutros. Cabe à equipa identificá-los, decidir como os tratar e avaliar se a melhoria de um degrada outro.

Ideias (indicativas)

As ideias seguintes resultam de uma análise preliminar da solução de base pela equipa docente. São **apenas indicativas**: não são obrigatórias, não esgotam as possibilidades e não definem requisitos. Outros requisitos não funcionais são aceites desde que bem fundamentados.

Requisito não funcional	**Ideia e situação na base**
Usabilidade — criação de pipelines	Tornar mais intuitiva a criação de pipelines, deixando claro a que tipos de *media* se aplica cada ferramenta e que encadeamentos são possíveis. *Na base:* num projeto com imagens e vídeos, algumas ferramentas deixam de estar disponíveis sem explicação, e um pipeline inválido só é detetado quando é executado.

Requisito não funcional	**Ideia e situação na base**
Manutenibilidade — modularidade do processamento de vídeo	Dividir o processamento de vídeo em vários microserviços, à semelhança do que já acontece com as ferramentas de imagem. *Na base:* todos os efeitos de vídeo, incluindo a legendagem automática e a extração de fotogramas, estão num único serviço.
Eficiência de desempenho — elasticidade	Ajustar dinamicamente os recursos de cada serviço à carga que lhe é destinada. *Na base:* os recursos são fixos e o processamento de vários vídeos tende a ser serializado; uma legendagem automática (reconhecimento de fala em CPU) ocupa o único serviço de vídeo e atrasa todos os outros trabalhos de vídeo.
Usabilidade — trabalho em segundo plano	Permitir que o utilizador continue a usar a aplicação enquanto um projeto é processado, e que acompanhe ou cancele esse processamento mesmo depois de recarregar a página. *Na base:* durante o processamento, um ecrã sobreposto bloqueia toda a aplicação; o progresso existe apenas no *browser* e perde-se com um *refresh*, deixando de ser possível acompanhá-lo ou cancelá-lo.
Eficiência de desempenho — carregamento de ficheiros	Tornar mais eficiente o processo de carregar ficheiros grandes. *Na base:* cada vídeo (até 1 GB) é mantido inteiro em memória em cada serviço que atravessa; não há indicação de progresso.
Segurança	Reforçar a confiança entre serviços internos, reduzir a superfície exposta e melhorar a gestão dos segredos da configuração. *Na base:* os serviços internos confiam uns nos outros sem verificação, alguns estão acessíveis diretamente a partir do exterior, a comunicação interna não valida certificados e os segredos estão no `docker-compose.yaml`, versionado no repositório.

O que se espera para cada requisito não funcional

1. **Diagnóstico.** Descrição da situação na solução de base, com evidência (excertos de código, componentes envolvidos, medições exploratórias) e do impacto para os *stakeholders*.
2. **Quantificação.** O requisito é especificado de forma verificável e indica, pelo menos: a escala (o que se mede e em que unidade), o método de medição (cenário, carga, ambiente, instrumento), o valor medido na base, o valor-alvo e o valor mínimo aceitável. Os valores devem ter origem identificável.
3. **Medição inicial (*baseline*).** Execução do método de medição sobre a solução de base, antes de qualquer alteração.
4. **Decisões arquiteturais e *refactoring*.** Alternativas consideradas, decisão tomada e consequências; reestruturação do código assistida por agentes de IA.
5. **Medição final e análise.** Repetição das medições, comparação com a *baseline* e com os valores-alvo, e análise crítica: o que melhorou, o que não foi atingido e porquê, e que outros requisitos não funcionais foram afetados.
6. **Preservação funcional.** Evidência, através de testes de regressão, de que a funcionalidade existente (imagem e vídeo) se mantém.

A estrutura do relatório e os entregáveis serão definidos numa versão seguinte deste documento.